

DineOS — Super Admin Guide

How the platform's Super Admin realm works: identity, login, tenant control, and feature licensing.

1. What problem the Super Admin solves

DineOS is not one restaurant's private backend — it is **one shared platform serving many client restaurants**. Each client restaurant is a **tenant**. The business plan is to sell DineOS to different restaurants at different price points by turning individual add-ons (Kitchen Display, Notifications, Billing, Real-time push) on or off per client.

The **Super Admin** is the platform operator's own account — you, not any restaurant's staff. It logs in separately from every restaurant's Admin/Manager/Server/Cashier accounts, and it is the only identity in the system that can:

- Create a new client restaurant (onboard a tenant) on the shared platform
- View every tenant that exists on the platform
- Flip a tenant's feature flags on/off live — no redeploy, no code change, no restart

2. Two separate identities, never merged

DineOS deliberately keeps two completely separate authentication realms instead of one user model with an "is_super_admin" flag:

	Restaurant staff	Super Admin
Model	<code>apps.authentication.User</code>	<code>apps.platform.PlatformAdmin</code>
Belongs to	Exactly one restaurant (tenant)	The platform itself — no restaurant
Login endpoint	<code>POST /v1/auth/login/</code>	<code>POST /platform/auth/login/</code>
Token claim carried	<code>restaurant_id</code> , <code>role</code>	<code>platform_admin: true</code>
Can see	Only its own restaurant's data	Every tenant's account record (name, flags, rates) — not their orders/menus/customers

Why full separation, not a flag? If Super Admin were just `User.is_super_admin = True`, every single tenant-scoping check in the codebase (every queryset filter, every permission class) would need to remember an "...unless super admin" exception. Miss updating just one of those checks six months from now, and a super admin session silently gains — or a bug silently grants ordinary staff — cross-tenant access. Making the two identities structurally different models, in different tables, authenticated by different classes, makes that class of bug impossible to introduce by accident rather than something that has to be remembered.

The PlatformAdmin model

```
class PlatformAdmin(AbstractBaseUser, PermissionsMixin):
    id = UUIDField(primary_key=True, default=uuid.uuid4)
    email = EmailField(unique=True)
    name = CharField(blank=True)
    is_active = BooleanField(default=True)
    is_staff = BooleanField(default=True)
    created_at = DateTimeField(auto_now_add=True)

    # own related_name on groups/user_permissions – required because
    # apps.authentication.User already claims the default related_name,
    # and Django forbids two user models sharing one.
```

Table: `platform_admins`. It has its own manager (`PlatformAdminManager`) with `create_user()` / `create_superuser()`, exactly like Django's normal user model — but it is *not* `settings.AUTH_USER_MODEL`. Django's built-in `ModelBackend` and default JWT authentication only ever look at `AUTH_USER_MODEL` (`authentication.User`), so a `PlatformAdmin` is architecturally invisible to the normal staff-login machinery. That is what forced building a small amount of custom login/auth code below — not a design preference, a consequence of wanting two real, separate models.

3. Logging in as Super Admin

- 1 **Request:** `POST /platform/auth/login/` with `{"email": ..., "password": ...}` — no auth header needed, this endpoint is `AllowAny`.
- 2 `PlatformLoginSerializer.validate()` looks up the `PlatformAdmin` row directly by email and calls `check_password()` itself — it does *not* go through Django's `authenticate()` / `ModelBackend`, because that backend is hardcoded to query `AUTH_USER_MODEL` and would never find a `PlatformAdmin` row at all.

- 3 `issue_platform_access_token(admin)` mints the JWT via `AccessToken.for_user(admin)` — deliberately *not* `RefreshToken.for_user()` or the standard `TokenObtainPairSerializer`. Those write a row to `OutstandingToken` (the token-blacklist app's bookkeeping table), which is foreign-keyed to `AUTH_USER_MODEL`. Handing it a `PlatformAdmin` instead throws a hard `ValueError`. Minting an access-only token sidesteps that table entirely — the trade-off is Super Admin sessions don't get a refresh token, so re-login is required once the access token expires. Given how infrequently platform-management actions happen, that trade-off was accepted rather than re-architecting the blacklist app.
- 4 The token is stamped with two custom claims: `platform_admin: true` and `name`. Critically, it carries **no** `restaurant_id` claim — that absence is what makes it structurally impossible for this token to be mistaken for a restaurant-staff session anywhere else in the system.
- 5 **Response:** `{"access": "<jwt>"}` — just the token. The Super Admin includes it as `Authorization: Bearer <token>` on every subsequent platform request.

How platform requests are authenticated

Every platform view uses `PlatformJWTAuthentication` instead of DRF's default JWT authentication:

```
class PlatformJWTAuthentication(JWTAuthentication):
    def get_user(self, validated_token):
        if not validated_token.get("platform_admin"):
            raise InvalidToken("Not a platform admin token.")
        admin = PlatformAdmin.objects.get(id=validated_token["user_id"])
        if not admin.is_active:
            raise InvalidToken("Platform admin account is inactive.")
        return admin
```

This gives two independent, structural rejections — neither relies on ID collisions being rare, both are guaranteed by the claim itself:

- **A restaurant staff token can never reach a platform endpoint.** Staff tokens never carry `platform_admin: true`, so `get_user()` raises immediately — before it even tries to look anyone up.
- **A platform token can never authenticate as restaurant staff.** Ordinary staff endpoints use DRF's default `JWTAuthentication`, which looks the token's `user_id` up against `AUTH_USER_MODEL` (`authentication.User`) only — a `PlatformAdmin`'s UUID does not exist in that table, so the lookup fails and the request is rejected.

Both directions are covered by automated tests (`tests/test_tenancy.py`):

`test_restaurant_staff_token_cannot_access_platform_endpoints` and `test_platform_token_cannot_authenticate_as_restaurant_staff`, both currently passing.

4. What the Super Admin can actually do — the endpoints

Method	Path	Purpose
POST	/platform/auth/login/	Log in, get an access token
GET	/platform/tenants/	List every client restaurant on the platform
POST	/platform/tenants/	Onboard a brand-new client restaurant
GET	/platform/tenants/{id}/	View one tenant's full record — name, slug, rates, all four flags
PATCH	/platform/tenants/{id}/	Update a tenant — most commonly, flip one feature flag
DELETE	/platform/tenants/{id}/	Remove a tenant record

All six require a valid Super Admin bearer token; all are backed by a single `TenantViewSet(ModelViewSet)`.

Onboarding a new client — worked example

Request

```
POST /platform/tenants/
Authorization: Bearer <platform token>
Content-Type: application/json

{
  "name": "Spice Route Grill",
  "slug": "spice-route-grill"
}
```

What happens server-side (`TenantViewSet.perform_create`):

- A new `Restaurant` row is created with that name/slug.
- All four feature flags default to `True` — a brand-new tenant starts with every add-on enabled.
- If the request didn't specify `gst_percentage` / `service_charge_percentage`, the platform-wide defaults from `.env` (`DEFAULT_GST_PERCENTAGE`, `DEFAULT_SERVICE_CHARGE_PERCENTAGE`) are applied automatically.

Response — `201 Created`, full `RestaurantSerializer` payload including the new tenant's `id`, ready to be used for creating that restaurant's first Admin staff account.

Selling a cheaper tier — flipping a flag

Request

```
PATCH /platform/tenants/{id}/  
Authorization: Bearer <platform token>  
Content-Type: application/json  
  
{ "kitchen_enabled": false }
```

Effect — takes effect on this tenant's very next order, no deploy, no restart:

- New orders skip the kitchen lifecycle entirely and are created already `SERVED` .
- The Kitchen Display Screen device for this tenant simply has nothing to show.
- Every other tenant on the platform is completely unaffected — the flag lives on the `Restaurant` row, not a global setting.

5. The four feature flags — what each one actually gates

Flag	When ON (default)	When OFF
<code>kitchen_enabled</code>	Orders go through the full lifecycle: <code>NEW</code> → <code>ACCEPTED</code> → <code>PREPARING</code> → <code>READY</code> → <code>COLLECTED</code> → <code>SERVED</code>	<code>place_order()</code> creates the order already <code>SERVED</code> — no kitchen ticket, no KDS involvement at all
<code>notifications_enabled</code>	<code>notify_role()</code> creates in-app notification rows for the relevant staff roles	<code>notify_role()</code> early-returns an empty list — nothing is written, nothing is pushed
<code>billing_enabled</code>	Bill preview and payment-confirmation endpoints work normally	<code>FeatureEnabledPermission("billing_enabled")</code> rejects both endpoints with <code>403</code> before any bill logic runs
<code>realtime_enabled</code>	WebSocket pushes (order updates, portion counts, payment confirmation) broadcast live to connected staff/customer clients	Every <code>_broadcast()</code> call site is a no-op, and all three WebSocket consumers (<code>Kitchen</code> / <code>Staff</code> / <code>Table</code>) reject <code>connect()</code> outright with close code <code>4003</code>

Each flag is checked at a different layer deliberately: `kitchen_enabled` and `notifications_enabled` branch inside the service-layer functions (so the behavior is identical no matter who called it — staff app, customer QR flow, or a future integration); `billing_enabled` is enforced at the permission-class layer (rejecting the whole request before touching business logic); `realtime_enabled` is enforced at both the broadcast call sites and at WebSocket connection time.

6. Where tenant resolution fits around the Super Admin

Every ordinary request in DineOS resolves a `request.tenant` — from a staff JWT's `restaurant_id` claim, from a Kitchen Display device's API key, or (for the login-less customer QR flow) explicitly inside the handful of views that need it. **Platform requests are the one exception:** a Super Admin token carries no `restaurant_id`, so `request.tenant` stays `None` for every platform call — which is correct, because `TenantViewSet` isn't scoped to one tenant at all; it operates across *all* tenants by design. The Super Admin is the one identity in the system that deliberately sits outside the tenant-scoping system, rather than being a tenant with elevated permission.

7. Creating or rotating the real Super Admin account

The one production Super Admin account is never created through the API or seeded with demo data — it's created by a dedicated management command that reads credentials from environment variables, so the password is never typed into a request body, a script, or a commit:

```
SUPERADMIN_EMAIL=owner@example.com SUPERADMIN_PASSWORD=<strong password> \  
python manage.py create_super_admin
```

The command is idempotent (`get_or_create` on email) — re-running it with a new `SUPERADMIN_PASSWORD` rotates the existing account's password instead of creating a duplicate. It never echoes the password back, and it refuses to run at all if either variable is missing.

The current real account is `info@krypsos.tech`. Its password isn't reproduced in this document on purpose — this file is meant to be shareable as a how-it-works guide without also being a credential.

8. The dev-only seed data, for contrast

`python manage.py seed_demo` creates a *separate, throwaway* platform admin (`super@dineos.platform / Demo@1234`) alongside two demo tenants — purely so automated tests and manual Postman runs can exercise cross-tenant isolation (two restaurants, two independent flag sets) without touching the real platform admin or any real client. It is intentionally kept in the codebase as an unused dev tool, never run against the live database.